around current solution, or probe a random direction, and move if we can get a better value.

Applying this to our graph problem, this means we would move a vertex in all directions, and each time compute the edge intersections of the new embedding.

And if this looks bad with a 2D cost function, remember that as the dimensionality of the search space grows we face the curse of dimensionality (see chapters 9-11), and we will have $2n$ parameters to tune for a graph with n vertices.

So, we might want to try a more efficient approach.

16.3 Gradient Descent

Why do we have to play “go fish” when we try local optimization? Remember that, for a graph with n vertices, it means we are in $2n$-dimensional space, so trying to move in every direction[10] randomly… is a lot!

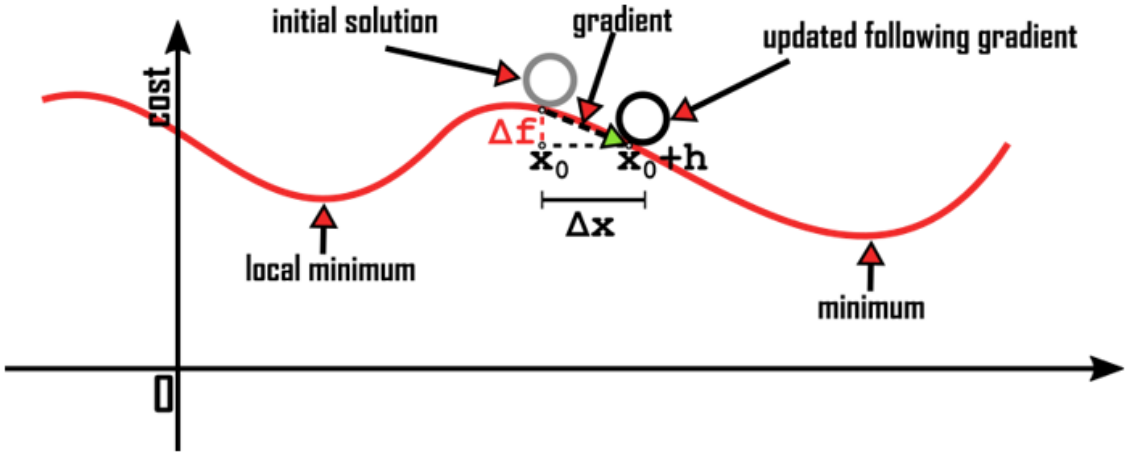


Figure 16.13 Gradient descent: Δf is the change in the cost function caused by a variation Δx in its parameter.